Algorithms Lab Record — Complete Collection

Includes: Algorithms, Pseudocode, Python Implementations, and Time Complexity

1. Bubble Sort

Problem statement: Sort an array by repeatedly swapping adjacent elements that are out of order.

Algorithm (step-by-step):

1. Start 2. Input array A with n elements 3. For i = 0 to n-1: For j = 0 to n-i-2: If A[j] > A[j+1], swap 4. Output sorted array 5. Stop

Pseudocode:

```
BUBBLE_SORT(A)
  n ← length(A)
  for i ← 0 to n-1
    for j ← 0 to n-i-2
      if A[j] > A[j+1]
        swap A[j], A[j+1]
```

Python Implementation:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

Time Complexity:

Best: O(n) (with early stop). Average/Worst: $O(n^2)$. Space: O(1).

2. Selection Sort

Problem statement: Sort an array by repeatedly selecting the minimum element from unsorted part and moving it to the beginning.

Algorithm (step-by-step):

1. Start 2. Input array A with n elements 3. For i = 0 to n-2: min_index = i for j = i+1 to n-1: if A[j] < A[min_index]: min_index = j swap A[i], A[min_index] 4. Output sorted array 5. Stop

Pseudocode:

```
SELECTION_SORT(A)
  for i ← 0 to n-2
    min_index ← i
    for j ← i+1 to n-1
      if A[j] < A[min_index]
        min_index ← j
    swap A[i], A[min_index]
```

Python Implementation:

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
```

```
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Time Complexity:

Best/Average/Worst: O(n²). Space: O(1).

3. Insertion Sort

Problem statement: Build a sorted array one item at a time by inserting elements into the correct position.

Algorithm (step-by-step):

1. Start 2. Input array A 3. For i = 1 to n-1: key = A[i] j = i-1 while j >= 0 and A[j] > key: A[j+1] = A[j] j = j-1 A[j+1] = key 4. Output sorted array 5. Stop

Pseudocode:

```
INSERTION_SORT(A)
  for i ← 1 to n-1
    key ← A[i]
    j ← i-1
    while j ≥ 0 and A[j] > key
      A[j+1] ← A[j]
      j ← j-1
    A[j+1] ← key
```

Python Implementation:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i-1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
```

Time Complexity:

Best: O(n). Average/Worst: O(n²). Space: O(1).

4. Job Scheduling (Activity Selection)

Problem statement: Given start[] and finish[] times of activities, select the maximum number of non-overlapping activities.

Algorithm (step-by-step):

1. Sort activities by finish time. 2. Select the first activity. 3. For each next activity i: if start[i] >= finish[last_selected]: select activity i 4. Output selected activities.

Pseudocode:

```
ACTIVITY_SELECTION(start, finish)
  n ← length(start)
  sort activities by finish time
  selected ← [0]
  last ← 0
  for i ← 1 to n-1
    if start[i] ≥ finish[last]
      selected.append(i)
      last ← i
  return selected
```

Python Implementation:

```
def activity_selection(start, finish):
    activities = sorted(zip(start, finish), key=lambda x: x[1])
    selected = []
    last_finish = -1
    for s, f in activities:
        if s >= last_finish:
            selected.append((s,f))
            last_finish = f
    return selected
```

Time Complexity:

Sorting: O(n log n). Selection scan: O(n). Total: O(n log n).

5. Coin Change (Greedy; Non-optimal in general)

Problem statement: Given coin denominations and an amount, find coins that sum to amount. Greedy works for canonical coin systems (e.g., USD) but not always.

Algorithm (step-by-step):

Greedy algorithm: 1. Sort coins in descending order. 2. For each coin: while amount >= coin: take coin, amount -= coin 3. If amount becomes 0, success; else no solution with greedy.

Pseudocode:

```
COIN_CHANGE(coins, amount)
  sort coins descending
  result ← []
  for coin in coins:
    while amount ≥ coin:
      amount ← amount - coin
      result.append(coin)
  if amount ≠ 0: report failure
```

Python Implementation:

```
def coin_change_greedy(coins, amount):
    coins = sorted(coins, reverse=True)
    res = []
    for c in coins:
        while amount >= c:
            amount -= c
            res.append(c)
    if amount != 0:
        return None  # greedy failed for arbitrary coins
    return res

# DP (minimum coins)
def coin_change_dp(coins, amount):
    INF = amount+1
    dp = [0] + [INF]*amount
    for i in range(1, amount+1):
        for c in coins:
            if c <= i:
                dp[i] = min(dp[i], dp[i-c]+1)
    return dp[amount] if dp[amount] <= amount else None
```

Time Complexity:

Greedy: O(m + k) where m=#coins sorted; practically O(m log m). DP: O(n*m) where n=amount, m=#coins.

6. Minimum Spanning Tree (Prim's and Kruskal's)

Problem statement: Connect all vertices in a weighted undirected graph with minimum total edge weight.

Algorithm (step-by-step):

Prim's (using min-heap): 1. Start from arbitrary vertex, mark visited. 2. Push edges from visited to heap by weight. 3. Pop smallest edge; if it connects to unvisited vertex, add to MST and mark visited; push its edges. 4. Repeat until all vertices visited. Kruskal's: 1. Sort all edges by weight. 2. Initialize DSU for vertices. 3. For each edge (u,v,w) in ascending order: if find(u) ≠ find(v): union(u,v); add edge to MST 4. Stop when MST has V-1 edges.

Pseudocode:

```
PRIM(G):
  choose start vertex s
  visited = {s}
  edges = min-heap of edges from s
  while heap not empty and |visited| < V:
    (w,u,v) = pop(heap)
    if v not in visited:
      add edge to MST
      visited.add(v)
      push edges from v to heap

KRUSKAL(G):
  sort edges by weight
  make-set for each vertex
  MST = []
  for (u,v,w) in edges:
    if find(u) ≠ find(v):
      union(u,v)
      MST.append((u,v,w))
```

Python Implementation:

```
import heapq

def prim_mst(adj, n):
    # adj: adjacency list {u: [(v,w), ...]}
    visited = [False]*n
    mst = []
    heap = [(0, 0, 0)]  # (weight, u, v) start from 0
    while heap and len(mst) < n-1:
        w,u,v = heapq.heappop(heap)
        if visited[v]:
            continue
        visited[v] = True
        if u != v:
            mst.append((u,v,w))
        for to,wt in adj.get(v,[]):
            if not visited[to]:
                heapq.heappush(heap, (wt, v, to))
    return mst

# Kruskal
class DSU:
    def __init__(self,n):
        self.p=list(range(n))
        self.r=[0]*n
    def find(self,x):
        if self.p[x]!=x:
            self.p[x]=self.find(self.p[x])
        return self.p[x]
    def union(self,a,b):
        ra,rb=self.find(a),self.find(b)
        if ra==rb: return False
        if self.r[ra]<self.r[rb]:
            self.p[ra]=rb
        else:
            self.p[rb]=ra
            if self.r[ra]==self.r[rb]:
```

```
                self.r[ra]+=1
        return True

def kruskal_mst(n, edges):
    edges.sort(key=lambda x: x[2])
    dsu=DSU(n)
    mst=[]
    for u,v,w in edges:
        if dsu.union(u,v):
            mst.append((u,v,w))
    return mst
```

Time Complexity:

Prim: O(E log V) with heap. Kruskal: O(E log E) dominated by sort; DSU ops ~α(n).

7. Minimum Platforms (Platform Selection)

Problem statement: Given arrival and departure times of trains, find the minimum number of platforms needed so no train waits.

Algorithm (step-by-step):

1. Sort arrival times and departure times. 2. Use two pointers i, j starting at 0. 3. If arrival[i] ≤ departure[j], platforms++, i++ else platforms--, j++ 4. Track max platforms seen. 5. Output max platforms.

Pseudocode:

```
MIN_PLATFORMS(arr, dep)
  sort arr, sort dep
  i = j = 0
  plat = maxplat = 0
  while i < n and j < n:
    if arr[i] ≤ dep[j]:
      plat += 1
      maxplat = max(maxplat, plat)
      i += 1
    else:
      plat -= 1
      j += 1
  return maxplat
```

Python Implementation:

```
def min_platforms(arr, dep):
    arr.sort()
    dep.sort()
    i = j = 0
    plat = maxplat = 0
    n = len(arr)
    while i < n and j < n:
        if arr[i] <= dep[j]:
            plat += 1
            maxplat = max(maxplat, plat)
            i += 1
        else:
            plat -= 1
            j += 1
    return maxplat
```

Time Complexity:

Sorting: O(n log n). Two-pointer scan: O(n). Total: O(n log n).

8. Set Cover (Greedy Approximation)

Problem statement: Given universe U and family of subsets S, select minimum subsets to cover U (NP-hard). Greedy gives ln(n)-approximation.

Algorithm (step-by-step):

Greedy algorithm: 1. While uncovered elements remain: select subset that covers the largest number of uncovered elements add it to cover, remove those elements from uncovered 2. Return selected subsets.

Pseudocode:

```
GREEDY_SET_COVER(U, S)
  uncovered = set(U)
  selected = []
  while uncovered:
    pick S_i that maximizes |S_i ∩ uncovered|
    selected.append(S_i)
    uncovered -= S_i
  return selected
```

Python Implementation:

```
def greedy_set_cover(universe, subsets):
    uncovered = set(universe)
    selected = []
    while uncovered:
        best = max(subsets, key=lambda s: len(s & uncovered))
        selected.append(best)
        uncovered -= best
    return selected
```

Time Complexity:

Greedy runs in O(m*n) per naive selection; with bookkeeping can be improved. Approximation factor: H_n ~ ln n.

9. Fractional Knapsack

Problem statement: Given items with values and weights and a capacity W, maximize total value; fractional items allowed.

Algorithm (step-by-step):

1. Compute ratio = value/weight for each item. 2. Sort items by decreasing ratio. 3. For each item in order: if weight ≤ remaining capacity: take whole item else take fraction to fill capacity and stop. 4. Output chosen (and value).

Pseudocode:

```
FRACTIONAL_KNAPSACK(items, W)
  compute ratio v_i/w_i for each item
  sort items by ratio desc
  total_value = 0
  for item in items:
    if W == 0: break
    take = min(item.w, W)
    total_value += take * (item.v/item.w)
    W -= take
  return total_value
```

Python Implementation:

```
def fractional_knapsack(items, W):
    # items: list of (value, weight)
    items = sorted(items, key=lambda x: x[0]/x[1], reverse=True)
    val = 0.0
    for v,w in items:
        if W == 0:
            break
        take = min(w, W)
        val += take * (v/w)
        W -= take
```

```
    return val
```

Time Complexity:

Sorting: O(n log n). Greedy scan: O(n). Total: O(n log n).

10. String Matching (Rabin-Karp & KMP)

Problem statement: Find occurrences of a pattern P of length m in text T of length n.

Algorithm (step-by-step):

Rabin-Karp: 1. Compute hash of P and first window of T. 2. Slide window; update rolling hash. 3. If hashes equal, verify characters. KMP: 1. Preprocess P to build LPS array. 2. Scan T with pointers i (text) and j (pattern); on mismatch use LPS to skip.

Pseudocode:

```
RABIN_KARP(T,P)
  d ← alphabet size, q ← prime
  hP ← hash(P), hT ← hash(T[0..m-1])
  for i = 0 to n-m:
    if hP == hT:
      if T[i..i+m-1] == P: report i
    if i < n-m:
      update hT by rolling hash
```

Python Implementation:

```
# Rabin-Karp
def rabin_karp(text, pattern, q=101):
    d = 256
    n, m = len(text), len(pattern)
    if m > n: return []
    h = pow(d, m-1, q)
    p = t = 0
    res = []
    for i in range(m):
        p = (p*d + ord(pattern[i])) % q
        t = (t*d + ord(text[i])) % q
    for i in range(n-m+1):
        if p == t:
            if text[i:i+m] == pattern:
                res.append(i)
        if i < n-m:
            t = (t - ord(text[i])*h) % q
            t = (t*d + ord(text[i+m])) % q
            t = (t + q) % q
    return res

# KMP
def kmp_lps(pat):
    m = len(pat)
    lps = [0]*m
    length = 0
    i = 1
    while i < m:
        if pat[i] == pat[length]:
            length += 1
            lps[i] = length
            i += 1
        else:
            if length != 0:
                length = lps[length-1]
            else:
                lps[i] = 0
                i += 1
    return lps
```

```
def kmp_search(txt, pat):
    n, m = len(txt), len(pat)
    lps = kmp_lps(pat)
    i = j = 0
    res = []
    while i < n:
        if txt[i] == pat[j]:
            i += 1; j += 1
            if j == m:
                res.append(i-j)
                j = lps[j-1]
        else:
            if j != 0:
                j = lps[j-1]
            else:
                i += 1
    return res
```

Time Complexity:

Rabin-Karp: Average O(n+m), Worst O(nm) (hash collisions). KMP: O(n+m).

11. Strassen's Matrix Multiplication

Problem statement: Multiply two n×n matrices faster than O(n^3) using divide-and-conquer (n power of 2 for simplicity).

Algorithm (step-by-step):

1. If n is small (1), return product. 2. Partition A and B into four (n/2)x(n/2) submatrices. 3. Compute 7 products M1..M7 using recursive calls and additions/subtractions. 4. Combine to form result quadrants. 5. Return resulting matrix.

Pseudocode:

```
STRASSEN(A,B):
  if n == 1: return A*B
  split A into A11,A12,A21,A22
  split B into B11,B12,B21,B22
  compute M1..M7 as per Strassen
  compute C11..C22 from M's
  combine C11..C22 into C
  return C
```

Python Implementation:

```
def add(A,B):
    n=len(A)
    return [[A[i][j]+B[i][j] for j in range(n)] for i in range(n)]

def sub(A,B):
    n=len(A)
    return [[A[i][j]-B[i][j] for j in range(n)] for i in range(n)]

def strassen(A,B):
    n = len(A)
    if n == 1:
        return [[A[0][0]*B[0][0]]]
    m = n//2
    A11 = [row[:m] for row in A[:m]]
    A12 = [row[m:] for row in A[:m]]
    A21 = [row[:m] for row in A[m:]]
    A22 = [row[m:] for row in A[m:]]
    B11 = [row[:m] for row in B[:m]]
    B12 = [row[m:] for row in B[:m]]
    B21 = [row[:m] for row in B[m:]]
    B22 = [row[m:] for row in B[m:]]
    M1 = strassen(add(A11,A22), add(B11,B22))
```

```
    M2 = strassen(add(A21,A22), B11)
    M3 = strassen(A11, sub(B12,B22))
    M4 = strassen(A22, sub(B21,B11))
    M5 = strassen(add(A11,A12), B22)
    M6 = strassen(sub(A21,A11), add(B11,B12))
    M7 = strassen(sub(A12,A22), add(B21,B22))
    def combine(C11,C12,C21,C22):
        new = []
        for i in range(len(C11)):
            new.append(C11[i] + C12[i])
        for i in range(len(C21)):
            new.append(C21[i] + C22[i])
        return new
    C11 = add(sub(add(M1,M4),M5),M7)
    C12 = add(M3,M5)
    C21 = add(M2,M4)
    C22 = add(sub(add(M1,M3),M2),M6)
    return combine(C11,C12,C21,C22)
```

Time Complexity:

Time: O(n^{log2 7}) ≈ O(n^{2.81}). Works best for large n and when using tuned implementations.

12. Karatsuba Multiplication (Large Numbers)

Problem statement: Multiply two large integers faster than O(n^2) by recursively splitting digits.

Algorithm (step-by-step):

1. If numbers are small, return product. 2. Split x into a*10^m + b, y into c*10^m + d. 3. Compute z0 = b*d, z1 = (a+b)*(c+d), z2 = a*c 4. result = z2*10^{2m} + (z1 - z2 - z0)*10^m + z0

Pseudocode:

```
KARATSUBA(x,y):
  if x < 10 or y < 10: return x*y
  m = max(len(x), len(y)) // 2
  split x -> a,b and y -> c,d
  z0 = KARATSUBA(b,d)
  z1 = KARATSUBA(a+b, c+d)
  z2 = KARATSUBA(a,c)
  return z2*10^{2m} + (z1-z2-z0)*10^{m} + z0
```

Python Implementation:

```
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y
    m = max(len(str(x)), len(str(y))) // 2
    pow10 = 10 ** m
    a, b = divmod(x, pow10)
    c, d = divmod(y, pow10)
    z0 = karatsuba(b, d)
    z1 = karatsuba(a + b, c + d)
    z2 = karatsuba(a, c)
    return (z2 * 10**(2*m)) + ((z1 - z2 - z0) * 10**m) + z0
```

Time Complexity:

Time: O(n^{log2 3}) ≈ O(n^{1.585}). More efficient for large integers compared to grade-school O(n^2).

End of Document — Algorithms Lab Record